Shubh Khandelwal

CS22B1090

MidSemReview2026

Document Details

Submission ID
trn:oid:::3618:129647647

Submission Date
Mar 2, 2026, 2:42 PM GMT+5:30

Download Date
Mar 2, 2026, 2:46 PM GMT+5:30

File Name
CS22B1090.pdf

File Size
156.6 KB

14 Pages

3,655 Words

19,994 Characters

10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Exclusions

- 4 Excluded Matches

Match Groups

34 Not Cited or Quoted 10%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

7% Internet sources
6% Publications
7% Submitted works (Student Papers)

Match Groups

- **34** Not Cited or Quoted 10%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

7% Internet sources
6% Publications
7% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	www.cnsr.ictas.vt.edu 2%
2	Internet	pdfs.semanticscholar.org 2%
3	Internet	core.ac.uk <1%
4	Internet	fliphtml5.com <1%
5	Student papers	King Saud University on 2016-08-29 <1%
6	Internet	www.irjaes.com <1%
7	Student papers	University of Sheffield on 2024-06-19 <1%
8	Publication	Lecture Notes in Computer Science, 2015. <1%
9	Student papers	University of Louisiana, Lafayette on 2017-04-29 <1%
10	Internet	innoovatum.com <1%

11 Student papers
Georgia Southwestern State University on 2021-04-26 <1%

12 Student papers
University of Glasgow on 2024-08-07 <1%

13 Internet
pdtn.org <1%

14 Publication
Xu Cheng, Jiangchuan Liu, Haiyang Wang. "Accelerating YouTube with video corr... <1%

15 Student papers
Rose-Hulman Institute of Technology on 2025-04-17 <1%

16 Student papers
University of Basrah - College of Science on 2021-04-01 <1%

17 Internet
assets-eu.researchsquare.com <1%

18 Internet
dokumen.pub <1%

19 Publication
"Information and Communications Security", Springer Science and Business Medi... <1%

20 Student papers
Higher Education Commission Pakistan on 2017-04-13 <1%

21 Student papers
Sardar Vallabhbhai National Inst. of Tech.Surat on 2017-08-04 <1%

22 Publication
Weiguo Lin, Jianfeng Ma, Teng Li, Haoyu Ye, Jiawei Zhang, Yongcai Xiao. "CrptAC: ... <1%

23 Publication
Zhangjie Fu, Xinle Wu, Chaowen Guan, Xingming Sun, Kui Ren. "Toward Efficient ... <1%

24 Student papers
Sardar Vallabhbhai National Inst. of Tech.Surat on 2017-01-17 <1%

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

1.1.1 Cloud Computing and Data Privacy

Cloud computing has revolutionized the way organizations deal with information management and processing by offering on-demand services for storage and computational resources. Through data outsourcing to the cloud, companies are able to save costs and simplify their information technology systems by relying on cloud service providers to deal with the physical layer of the cloud. However, this new way of dealing with data has resulted in critical security concerns regarding data confidentiality. To prevent the risk of data breach by curious cloud service providers, data owners tend to encrypt data such as medical records and financial logs before outsourcing them.

Although this encryption method guarantees data privacy, it poses a major hindrance to data retrieval as well as utilization. In the past, retrieving certain data from the encrypted data repository has called for the entire data repository to be downloaded and decrypted. This practice becomes absolutely impossible as the volume of data increases. This is why Searchable Encryption (SE) has come to play a crucial role as a data security technology that allows users to carry out keyword search operations directly on the encrypted data within the cloud environment [1].

1.2 Limitations of Existing Approaches

Ever since the development and introduction of early searchable encryption systems, researchers have sought to improve the systems with regard to search, efficiency, and privacy. The early systems were only able to perform either a single or multi-keyword

search [1]. The early systems, which were very strict, did not have the ability to perform a search with a typo, and a user would receive zero results for a keyword that had a spelling error [1]. The later versions of searchable encryption systems included the option for a fuzzy search, which allowed for a few typos. However, the systems had a major problem with regard to storage, and a large number of index sizes had to be generated for a keyword with a specified edit distance [2]. The later versions of searchable encryption systems removed the storage problem by incorporating Locality-Sensitive Hashing (LSH), which allowed for a fuzzy search [3]. However, the later versions of searchable encryption systems, which were based on LSH, had major drawbacks.

Table 1.1: Evolution of Searchable Encryption Paradigms

Scheme	Search Type	Cryptography	False Negatives
Early SE [1]	Exact Match	Symmetric	N/A
Expanded Index [2]	Single Fuzzy	Symmetric	Low (High Storage)
Baseline LSH [3]	Multi Fuzzy	Symmetric	High
Proposed MP-LSH	**Multi Fuzzy**	**Asymmetric**	**Low**

The primary limitations of the current LSH-based symmetric schemes include:

- **Elevated False Negative Rates:** Standard LSH algorithms inherently produce false negatives by missing relevant documents that fall just outside strict hashing boundaries.
- **Symmetric Key Management Bottlenecks:** Relying on symmetric matrices for inner-product computation requires sharing a master secret key with all authorized users. In a dynamic, multi-user cloud environment, this makes key distribution, rotation, and revocation virtually impossible to manage securely.
- **Rigid Access Control:** The symmetric nature of the baseline scheme prevents the data owner from granting granular, user-specific search permissions without compromising the entire system's security architecture.

1.3 Proposed Solution and Contributions

To address the inflexibility of exact matching algorithms and the scalability limitations of symmetric key systems, this project introduces an efficient and sophisticated public key-based multi-keyword fuzzy search paradigm. The proposed architecture provides an exact fuzzy search via algorithmic advances while completely bypassing the need for index expansion and dictionaries, making our solution computationally efficient [3].

The key defining feature of the proposed system is its shift from a symmetric matrix-based search evaluation process to an Inner Product Predicate Encryption (IPE) paradigm [4]. This allows for an asymmetric data security solution wherein a number of data providers can use a commonly available public key for encrypting the indexes of documents, while the data owner maintains a master secret key for issuing and revoking specific search tokens for individual users. To further improve the search accuracy, the proposed solution has also upgraded the traditional hashing process with a Multi-Probe Locality-Sensitive Hashing (MP-LSH) [5]. By evaluating adjacent hash buckets that have a high probability of containing valid nearest neighbors, the solution has significantly reduced the false negative rate.

CHAPTER 2

SCHEME MODEL

2.1 System Architecture

The proposed asymmetric searchable encryption scheme includes mainly three entities, namely, the Data Owner (DO), the Cloud Server (CS), and the Users. Unlike the traditional symmetric-based schemes, which often involve complex key distribution issues, the proposed scheme exploits the power of asymmetric cryptography.

2.1.1 System Entities

- **Data Owner (DO):** The DO is responsible for generating the system's cryptographic keys, specifically a Public Key (PK) and a Master Secret Key (MSK). The data owner utilizes the PK to generate secure Bloom filter indexes and encrypt the files prior to uploading them to the cloud environment. Crucially, the DO acts as the central authority that uses the MSK to grant search capabilities to authorized users by generating secure search tokens.
- **Cloud Server (CS):** The CS is an entity with abundant storage and computational resources. It serves as the repository for the ciphertexts and their corresponding searchable indexes. Upon taking in a user's search token, the server runs the IPE evaluation routine against the index structures to identify and retrieve the appropriate documents.
- **Users:** Users are entities authorized by the DO to perform keyword searches over the outsourced encrypted data. To trigger a retrieval request, an authorized individual maps their search terms to a query vector, obtains a specific search token from the DO, and forwards this token to the storage server.

2.1.2 Operational Flow

The workflow of the proposed scheme operates in the following phases:

1. **Setup:** The DO initializes the IPE system parameters, publishing the PK and securing the MSK.

2. **Data Outsourcing:** The DO isolates terms from every file, maps them to bigram arrays, and populates a Bloom filter utilizing LSH functions [3].This filter is then secured with the PK, creating the final index that accompanies the encrypted file to the CS.

3. **Trapdoor Generation:** An authorized user wishes to search for a set of keywords. The keywords are transformed into a query vector and sent to the DO. The DO applies the MSK to generate a secure search token (TK_Q) and returns it to the user.

4. **Secure Search via MP-LSH:** The user submits TK_Q to the CS. The CS applies the IPE evaluation algorithm [4] to compute the inner product between the token and the stored encrypted indexes. To resolve the high false negative rate of standard LSH, the CS utilizes Multi-Probe LSH (MP-LSH) logic [5], systematically checking adjacent hash buckets to locate fuzzy matches that fall slightly outside the strict hashing boundaries. The CS then returns the matching encrypted files.

2.2 Security Model

We consider the server to be "honest but curious" with respect to the service provider.With these assumptions, the server reliably adheres to its operational protocols and provides data availability.However, the server plays the role of a passive eavesdropper, which might allow it to examine the ciphertexts, indexes, and queries to gain insights into the confidential file information or the behavior of users searching for the files.

In order to thoroughly test the privacy of the system, we consider two types of adversaries based on the level of intelligence they possess:

- **Known Ciphertext Model:** The adversary is restricted to viewing the outsourced ciphertexts, the encrypted index vectors, and the submitted search tokens.The server tracks access patterns—noting which tokens yield which files—but remains ignorant of any outside contextual information.
- **Known Background Model:** The adversary is equipped with supplementary statistical knowledge about the outsourced collection, including term frequencies or historical distributions derived from comparable plaintext archives.The adversary attempts to use this statistical knowledge to launch linear analysis attacks against the secure indexes to deduce the underlying keywords.

2.3 Design Goals

To successfully overcome the hurdles of encrypted search in shared ecosystems, our asymmetric MP-LSH architecture pursues a distinct set of operational and security targets:

- **Multi-Keyword Fuzzy Search:** The architecture must handle complex, multi-term queries and gracefully manage typographical mistakes, guaranteeing that relevant files are surfaced despite minor spelling anomalies.
- **High Result Accuracy (Recall):** Through the adoption of MP-LSH [5], our design seeks to dramatically lower the instances of missed matches inherent to basic LSH systems, maximizing retrieval recall.
- **Robust Privacy Guarantees:** Our framework guarantees the protection of sensitive file contents and search terms.The storage provider remains incapable of extracting plaintext terms from the stored indexes or queries (Keyword Privacy), and it is prevented from linking distinct tokens that represent identical search terms (Trapdoor Unlinkability).
- **Dynamic Updates Without Dictionaries:** Moving away from legacy systems that depend on rigid, global vocabulary lists, our architecture enables fluid data modifications—such as insertions, deletions, or edits—without requiring a full reconstruction of the searchable index.

CHAPTER 3

MATHEMATICAL PRELIMINARIES

3.1 Keyword Transformation: Bigram Vectors

For the purpose of quantitative distance computation between textual strings, each keyword must be transformed into a numerical vector format. First off, a keyword is transformed into a bigram set, where every combination of two consecutive letters within a keyword is considered [3].For example, a keyword "network" is transformed into a bigram collection ne, et, tw, wo, or, rk [3].

This bigram collection is further transformed into a definitive vector of dimensionality 26^2, where each dimension of the vector represents a unique combination of two consecutive letters within a keyword.A dimension of the vector is set to 1 if the bigram is included within the keyword's bigram collection; otherwise, it is set to 0.Since the order of bigrams is ignored during this transformation, this vectorization inherently resists positional typos as well as character substitution typos.As a result of this transformation, any keyword spelling error is guaranteed to produce a vector that is geometrically adjacent to the correct spelling of the keyword, thus enabling a precise evaluation of their similarity through Euclidean distance computation [3].

3.2 Bloom Filters

A Bloom filter acts like a memory-optimized probabilistic array, which is best suited for quick membership tests. A Bloom filter consists of a bit array of size m, which is initialized to all zeros. For adding an element a to the dataset $\mathcal{S} = \{a_1, a_2, \ldots, a_n\}$, the structure makes use of a family of l different hash functions, which are defined as $\mathcal{H} = \{h_i | h_i : \mathcal{S} \rightarrow [1, m], 1 \leq i \leq l\}$. Each function h_i maps the element a to some integer $h_i(a)$ within the range $[1, m]$, and the corresponding elements at the locations $h_i(a)$ are flipped to 1.

In order to test the presence of a particular element q, the same family of l hash functions is applied. If the corresponding elements at the locations $h_i(q)$ contain a zero, the particular element is guaranteed to be absent. On the other hand, if all the checked indices report a 1, then the element is either present or a false positive has occurred.In the case of an m-bit array and l functions, the probability of a false positive is approximately given by $(1 - e^{-\frac{ln}{m}})^l$.

3.3 Locality-Sensitive Hashing (LSH)

While it is the purpose of cryptographic hash functions to produce hash digests that are vastly dissimilar even for almost identical input data, it is the very purpose of Locality Sensitive Hashing to find an LSH function that maps spatially proximal data points to hash bins with the same hash signature with a significantly higher probability than spatially distant points [6].

Mathematically, assuming a distance metric $d(s,t)$ between two vectors, a designated hash family $\mathcal{H}$ qualifies as (r_1, r_2, p_1, p_2)-sensitive if it meets these probabilistic conditions:

$$\text{if } d(s,t) \leq r_1 : Pr[h(s) = h(t)] \geq p_1$$

$$\text{if } d(s,t) \geq r_2 : Pr[h(s) = h(t)] \leq p_2$$

For mapping the bigram vectors in our architecture, we deploy the p-stable LSH family [6].The specific hash function is formulated as:

$$h_{a,b}(v) = \lfloor \frac{a \cdot v + b}{w} \rfloor$$

In this formulation, v and a represent vectors, while b and w are scalar real numbers acting as projection parameters.

3.4 Multi-Probe LSH (MP-LSH)

Though in the conventional LSH, all the vectors are mapped to the same bucket, in many cases, the vectors near the boundary of the projection width w are mapped to the next bucket, resulting in false negatives. To address this, we propose a method to improve the conventional LSH by incorporating a novel approach called Multi-Probe Locality-Sensitive Hashing (MP-LSH) [5].

In the conventional LSH, instead of using the hash value of the main function $h_{a,b}(v)$, MP-LSH proposes a set of perturbation vectors $c = (\delta_1, \delta_2, \ldots, \delta_l)$. These perturbation vectors are applied to the conventional hash function output to identify the adjacent buckets $h(v) + c$ that have the maximum probability of containing the nearest neighbors [5].

3.5 Bilinear Pairings and Inner Product Predicate Encryption

To transition from the symmetric matrix multiplication of the baseline scheme to a public-key infrastructure, we employ Bilinear Pairings to facilitate Inner Product Predicate Encryption (IPE) [4]. Let $\mathbb{G}$ and $\mathbb{G}_T$ be two cyclic multiplicative groups of prime order p, and let g be a generator of $\mathbb{G}$. A bilinear map $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$ possesses the following properties:

- **Bilinearity:** For all $u, v \in \mathbb{G}$ and $a, b \in \mathbb{Z}_p^*$, $e(u^a, v^b) = e(u, v)^{ab}$.
- **Non-degeneracy:** $e(g, g) \neq 1$.
- **Computability:** There exists an efficient algorithm to compute $e(u, v)$ for any $u, v \in \mathbb{G}$.

Utilizing this mapping, the IPE framework enables the Cloud Server to "blindly" compute the inner product between $\langle \mathcal{I}_\mathcal{D}, \mathcal{Q} \rangle$, as proposed by Shen et al. in [4]. The Cloud Server matches each component of the index's ciphertext ($CT_\mathcal{I}$) with the user's trapdoor token ($TK_\mathcal{Q}$). The bilinearity of the IPE enables the two vectors to interact and produce the matched LSH bits count.

CHAPTER 4

DETAILED SYSTEM IMPLEMENTATION

In this chapter, we describe the specific algorithmic instantiation of the proposed Asymmetric Multi-Probe LSH SE scheme. Specifically, the scheme is formalized by four main polynomial-time algorithms: **Setup**, **BuildIndex**, **Trapdoor**, and **Search**.

4.1 System Setup (Setup)

The Setup phase is executed by the Data Owner (DO) to initialize the cryptographic parameters of the Inner Product Predicate Encryption (IPE) framework and the public hashing functions.

- **Input:** A security parameter λ, and the desired Bloom filter length m.
- **Operations:**
  1. The DO initializes the bilinear pairing parameters $(\mathbb{G}, \mathbb{G}_T, p, e, g)$ where p is a prime based on λ [4].
  2. The DO generates the Master Secret Key (MSK) and the corresponding Public Key (PK) required for the IPE scheme over vectors of length m.
  3. The DO selects l independent LSH functions from the p-stable LSH family $\mathcal{H} = \{h : \{0,1\}^{26^2} \to \{0,1\}^m\}$ [6].
- **Output:** The DO keeps MSK completely private. The DO publicly broadcasts the system parameters $PARAMS = \{PK, \mathbb{G}, \mathbb{G}_T, e, \mathcal{H}, m, l\}$.

4.2 Index Generation (BuildIndex)

Before outsourcing a document, the Data Owner (or an authorized data provider holding the PK) must generate its secure index.

- **Input:** A document $\mathcal{D}$, the public key PK, and the hashing parameters.
- **Operations:**
  1. **Keyword Extraction:** Extract the set of distinct keywords $\mathcal{W}_\mathcal{D} = \{w_1, w_2, \dots\}$ from the document $\mathcal{D}$.

2. **Vectorization:** Convert each keyword w_i into its 26^2-dimensional bigram vector representation [3].
3. **Bloom Filter Insertion:** Initialize an m-bit Bloom filter $\mathcal{I}_\mathcal{D}$ to all zeros. For each bigram vector, apply the l LSH functions $h_j \in \mathcal{H}$ (where $1 \leq j \leq l$) and set the corresponding bits in $\mathcal{I}_\mathcal{D}$ to 1.
4. **Asymmetric Encryption:** Treat the final Bloom filter $\mathcal{I}_\mathcal{D}$ as an m-dimensional vector. Encrypt this vector using the IPE encryption algorithm with the Public Key (PK) to produce the ciphertext index $CT_\mathcal{I}$ [4].

- **Output:** The secure index $CT_\mathcal{I}$, which is outsourced to the Cloud Server alongside the symmetrically encrypted document payload.

4.3 Search Token Generation (Trapdoor)

When an authorized user wants to search for documents, they cannot generate the search token themselves because they do not possess the MSK. They must interact securely with the DO.

- **Input:** The user's query containing fuzzy keywords $\mathcal{Q}_{words}$, and the DO's MSK.
- **Operations:**
  1. **Query Formulation:** The user transforms the keywords in $\mathcal{Q}_{words}$ into bigram vectors and maps them into an m-bit Bloom filter query vector $\mathcal{Q}$ using the same l public LSH functions $h_j \in \mathcal{H}$.
  2. **Token Request:** The user securely transmits the plaintext vector $\mathcal{Q}$ (or the raw keywords, depending on the trust model) to the Data Owner.
  3. **Token Generation:** The DO utilizes the Master Secret Key (MSK) and the IPE token generation algorithm to encrypt the query vector $\mathcal{Q}$ into a secure search token $TK_\mathcal{Q}$ [4].
- **Output:** The authorized trapdoor token $TK_\mathcal{Q}$, which the user submits to the Cloud Server.

4.4 Secure Search and MP-LSH Evaluation (Search)

Upon receiving the token from the user, the Cloud Server evaluates the match against the stored encrypted indexes without learning the plaintext keywords or index contents.

- **Input:** The user's trapdoor token $TK_\mathcal{Q}$, the encrypted document indexes $CT_\mathcal{I}$, and the Multi-Probe perturbation parameter c.

- **Operations:**
  1. **Primary Evaluation:** For each encrypted index $CT_{\mathcal{I}}$ in the database, the CS executes the IPE public evaluation algorithm $e(CT_{\mathcal{I}}, TK_{\mathcal{Q}})$. Due to the bilinear pairings, this yields the inner product $\langle \mathcal{I}_{\mathcal{D}}, \mathcal{Q} \rangle$, representing the number of matched bits [4].
  2. **Multi-Probe Expansion:** To minimize false negatives for heavily misspelled queries, the CS generates systematic perturbation vectors $c = (\delta_1, \ldots, \delta_l)$ [5].
  3. **Adjacent Bucket Checking:** The CS mathematically shifts the query token's evaluation space by the perturbation vectors c to check adjacent hashing buckets. If an adjacent bucket yields a higher inner product score, the CS dynamically updates the similarity rank.
  4. **Ranking:** The CS sorts the documents based on the final, probe-adjusted inner product scores.

- **Output:** The CS returns the top-k ranked encrypted documents to the querying user.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

In this report, we addressed the challenging task of multi-keyword fuzzy search over encrypted cloud data [3].Although the symmetric cryptographic primitives had been successfully used for the basic keyword search, the exact keyword spellings and the traditional symmetric key distribution model had become the main obstacles for the real-world application of the symmetric primitives [1, 2].The baseline analysis was conducted on the new paradigm, which employed the LSH functions with the assistance of the Bloom filters for the development of the secure document indexes [3, 7].By utilizing the Euclidean distance for the modeling of the keyword similarity based on the bigram vectors, the baseline scheme enabled the multi-keyword fuzzy matching without the exponential increase in the size of the index [3, 6].

However, to mitigate the inherent false negative limitations of conventional LSH, as well as the key management limitations of symmetric cryptography, this project has proposed a significantly enhanced architecture. By migrating the underlying matching algorithm from a set of symmetric matrix operations to a set of Inner Product Predicate Encryption (IPE) operations, the proposed scheme enables data owners to securely outsource data via a universally accessible public key, as well as dynamically delegate search privileges to users via a master secret [4].

In addition, the integration of MP-LSH significantly enhances the overall accuracy of data retrieval by systematically examining adjacent, highly probable hash buckets, thereby ensuring that relevant data, even in the presence of minor typographical errors, are not discarded [5]. Ultimately, the proposed scheme provides a highly accurate, scalable, and secure data search environment.

5.2 Future Scope

While the proposed Asymmetric MP-LSH scheme significantly advances the state-of-the-art in secure data retrieval, several avenues remain open for future research and optimization:

- **Integration of TF-IDF for Relevance Ranking:** The current inner product evaluation approach computes the raw count of matching LSH bits [3].The future iterations can include the TF-IDF weights into the secure representations. This would enable the Cloud Server to rank the results based on the relevance of the documents and the significance of the keywords.
- **Optimized Dynamic Data Updates:** Although the existing approach eliminates the need to maintain a predefined global dictionary [3],in the future, the focus should be on developing a highly optimized protocol for performing dynamic data updates, including insertions, deletions, and modifications, in the context of the Asymmetric IPE scheme. The aim is to perform these operations with low computational overhead without decrypting or re-indexing the surrounding untouched data.
- **Semantic and Natural Language Search:** While the current bigram vectorization approach very elegantly solves the typographical error problem, it does nothing for the ability to actually understand the concepts. Expanding the system to incorporate the capability for real semantic search, via the incorporation of secure word embeddings (such as Word2Vec), would be a significant step forward. This would allow the system to search for synonymous concepts (such as "automobile" and "car") directly on the encrypted data.
- **Hardware-Accelerated Cryptographic Evaluation:** The computational complexity of the bilinear pairing operations for evaluating the IPE for large data sets is a concern. As a future direction, hardware acceleration for the Cloud Server-side, possibly with GPUs or other hardware accelerators, might be explored for significantly reducing the search latency for large data sets.